

Display Linkage: a Z Specification

Formal model of the Hub’s hold/reconcile/never-control discipline

September 2026

1 Introduction

Closing the Lux window used to black-hole content: an idle, no-display Hub retried at a fixed, tight cadence forever, because the code could not tell “no display has ever been connected” apart from “a connected display just misbehaved,” and both conditions advanced the same short backoff. The corrected design (`docs/architecture/display-presence-demand-driven.md`) holds content safely in `HubDisplay` (unchanged), reconciles it the moment a display connects (unchanged, DES-068), and fixes the one thing that was actually broken: the retry *cadence* when nothing is connected at all, and the coordination between the Hub’s two independent threads that each dial the display.

Why a new specification, not an extension of `hub_replicator.tex`. `docs/hub_replicator.tex` already proves six invariants for the Hub’s *single* background worker sending to *one* display: no second writer, agent-path liveness, no lock cycle between the store lock and the client lock, no lost update across a reap or reconnect, no two live displays, and menu durability. That model has exactly one thread that ever touches the display connection. This design’s entire novelty is a *second* thread — `DisplayLiveness` — dialing the *same* client registry concurrently, coordinated with the replicator through a shared `threading.Event` that is deliberately consulted *without* holding the registry’s lock, plus two independently-tuned exponential backoff curves that must never be conflated. None of that concurrency shape, nor any of its invariants, exists in `hub_replicator.tex`’s `Repl` schema. Grafting a second actor onto an already-verified, committed regression artifact would force re-deriving its six proven invariants under an interleaving they were never checked against, for invariants that are otherwise orthogonal to them. A fresh, narrower specification — cross-referencing `hub_replicator.tex` for the reconciliation hook it already models (`Ensure/Remark/Reconn`’s re-mark, the concrete counterpart of this design’s DES-068 connect-success reconcile) — is more faithful to what changed and leaves the existing proof untouched. (The mission brief cites this as “§10 item 13”; the design document’s own numbering makes it item 13’s cross-reference to item 11, the test list — noted here since the two numbers do not otherwise agree.)

Scope. This model covers the connect/disconnect/backoff mechanism newly introduced by the design: `DisplayLinkage` as an observed classification of (`connected`, `live content`); the two concurrent actors dialing `ClientRegistry`; the disconnected backoff (2s → 120s in the design, abstracted here to a small bounded step count) kept structurally separate from the wedged-display backoff (0.1s → 2.0s, likewise abstracted); and the shared reconnect signal with its lock-freedom property. It does *not* re-derive the fine-grained send-cycle mechanics (draining, snapshotting under a store lock, per-scene blank-on-empty) that `hub_replicator.tex` already covers: one “send cycle” here is modelled as a single atomic step that delivers whatever is currently held, which is adequate for every invariant below and is stated plainly so the simplification is never mistaken for silence.

Two further simplifications are implementer-relevant rather than merely notational, and are called out here so neither is read as a claim about where the real reset or the real wake happens.

The disconnected-backoff reset point. This model resets `discBackoff` to 0 at `RDialOk` — the dial succeeding — because a dial and a send are one atomic step here. Design §6.3 resets `_disconnected_backoff` at a *clean send* (`outcome == "clean"`), one step later than the dial in the real two-phase cycle. No invariant above is sensitive to which of the two points is used: under a sustained disconnection `RDialOk` never fires at all, so the reset in either place is equally inert while the property I4 actually exercises (the climb) is underway. The *implementation* must follow §6.3’s clean-send reset, not this model’s dial-success

reset — a dial that succeeds but whose immediate send then fails should not reset the disconnected back-off, and this model’s coarser single-step cycle cannot distinguish that case from one where the send also succeeds.

The wait/clear lost-wakeup window. The real `wait_for_reconnect` body is `woke = self._reconnected.wait(timeo`
`self._reconnected.clear(); return woke` — two statements, with a window between the event’s own wake and the unconditional `.clear()` in which another thread could, in principle, call `.set()` again and have that second signal silently discarded. `RWaitWoken` and `RWaitTimeout` each model the whole body as one atomic step, so this window does not exist in the carrier and this model proves nothing about it either way. The consequence of a real lost wakeup here is bounded — the replicator does not learn of the second signal until its *next* dial attempt, which is bounded by the very backoff delay I4 already proves reaches its cap and no further, so this degrades to added retry latency, never to a stuck wait or lost content. It is out of this model’s scope by construction, not by oversight, and an implementer reviewing the real `wait_for_reconnect` body should confirm that bound holds of the shipped code.

Eight properties, drawn directly from §12 of the design document, must hold across every interleaving of the two actors and the content-owning agents:

- I1 — content never lost while disconnected.** Every scene that is live is either queued for delivery or already known to be shown; no interleaving of push, disconnect, and retry silently drops it.
- I2 — reconcile exactly once.** The moment either actor’s dial turns a disconnected registry into a connected one, every currently live scene is marked for redelivery in that same step — never missed, never re-queued twice for one reconnect.
- I3 — neither actor blocks or spins on a missing display.** Every dial resolves in one bounded step; the replicator’s retry interval is always the disconnected backoff’s current value; the prober’s interval is always one of exactly two named speeds.
- I4 — the disconnected backoff is genuinely exponential and reaches a slow steady state.** Under a sustained disconnection its own counter climbs to its own, higher cap — not merely bounded, but bounded at the *right*, slower ceiling.
- I5 — the Hub never issues a process-control call to the display.** No Hub-side operation ever causes a display to become reachable; only the environment does.
- I6 — the early-wake mechanism is present, reachable, and load-bearing.** The replicator’s wait *can* be broken by the shared event rather than only by the full delay elapsing, and a blocking sleep in place of that mechanism destroys the property. “Within one `DisplayLiveness` probe tick,” the design’s own real-time phrasing, is a timing claim an untimed model cannot discharge; see the invariant’s own paragraph (§10) for exactly what is and is not proven.
- I7 — a disconnected cycle never advances the wedged-display backoff, and vice versa.** The two counters are structurally disjoint.
- I8 — deadlock-freedom, with the prober modelled as a genuine second actor.** In particular, the specific claim to check: the replicator’s wait for a reconnect never holds the registry’s lock.

2 Basic Types

[*SCENE*]

SCENE is the set of scenes `HubDisplay` can hold live; one suffices for every invariant below except I2’s “every currently live scene” claim, which needs at least two to show that a batch reconcile covers more than a singleton. Bounded by `ProB`’s `DEFAULT_SETSIZE`.

3 Free Types

FLAG ::= *set* | *clear*

LOCK ::= *held* | *free*

Two-valued markers, following `hub_replicator.tex`'s own convention: `FLAG` for the connected belief, the shared reconnect signal, and the “woken early or timed out” witness; `LOCK` for `ClientRegistry._lock`.

$$RPHASE ::= ridle \mid rwaitdisc \mid rsendng \mid rhealing$$

The replicator's own program counter, restricted to the phases this design adds or changes. `ridle`: no cycle in flight, about to dial. `rwaitdisc`: parked in `_wait_disconnected`, having just failed to connect at all. `rsendng`: connected, about to deliver whatever is held. `rhealing`: a connected-but-misbehaving delivery is being reaped/reconnected — the machinery `docs/hub_replicator.tex` already proves in full; here it is one atomic step, entered and left.

$$SPEED ::= pconn \mid pdisc$$

`DisplayLiveness`'s current probe pacing: `pconn`, the normal connected-interaction cadence, or `pdisc`, the slightly relaxed `_DISCONNECTED_PROBE_INTERVAL` — the two, and only two, named values §6.4 defines.

4 State

The state is flat: one schema, exactly as `docs/display_lifecycle.tex` and `docs/hub_replicator.tex` require for ProB. Its predicate carries only structural bookkeeping — facts true by construction of every operation below, correct or buggy — never one of the eight safety properties themselves, which must stay free to be violated by a faulty variant or the model would prove nothing.

<i>Link</i>	
<i>live</i> : $\mathbb{P} \text{ SCENE}$	
<i>dirty</i> : $\mathbb{P} \text{ SCENE}$	
<i>shown</i> : $\mathbb{P} \text{ SCENE}$	
<i>linked</i> : <i>FLAG</i>	
<i>reconnected</i> : <i>FLAG</i>	
<i>wokenEarly</i> : <i>FLAG</i>	
<i>reachable</i> : <i>FLAG</i>	
<i>sendOk</i> : <i>FLAG</i>	
<i>regLock</i> : <i>LOCK</i>	
<i>rphase</i> : <i>RPHASE</i>	
<i>discBackoff</i> : $\mathbb{N}$	
<i>wedgeBackoff</i> : $\mathbb{N}$	
<i>Interval</i> : <i>SPEED</i>	
<i>dirty</i> $\subseteq$ <i>live</i>	
<i>shown</i> $\subseteq$ <i>live</i>	

`live` is the set of scenes currently live in `HubDisplay` — the content-owning agents' own state, touched only by the two content operations below, never by either dialing actor. `dirty` is the set still owed to the display: exactly the design's “content is held” set, populated by a push and by the reconcile-on-connect, and drained only by an actual successful delivery. `shown` records which live scenes are believed already delivered — the abstraction standing in for `hub_replicator.tex`'s own `displayed` map, coarsened to a set because this spec does not re-derive per-scene send mechanics.

`linked` is `ClientRegistry`'s own belief about whether a display is connected — what `DisplayLinkage.classify` reads as its `connected` argument, together with `#live` for its `live_scene_count`. `DisplayLinkage` itself is not stored: consistent with §4 of the design (“no new stored state, because there is nothing for the Hub to decide here, only something to observe”), it is a derived classification of (`linked`, `live`):

$$\begin{aligned} \text{Linkage}(\text{linked}, \text{live}) &\triangleq \{ \text{DISCONNECTED} \\ &\text{linked} = \text{clear} \wedge \text{live} = \emptyset \\ &\text{HELD} \\ &\text{linked} = \text{clear} \wedge \text{live} \neq \emptyset \\ &\text{CONNECTED_IDLE} \\ &\text{linked} = \text{set} \wedge \text{live} = \emptyset \\ &\text{CONNECTED_ACTIVE} \\ &\text{linked} = \text{set} \wedge \text{live} \neq \emptyset \end{aligned}$$

exactly `DisplayLinkage.classify`'s own four-way case split. No invariant below needs to name a `Linkage` value directly; each is stated over `linked`, `live`, `dirty`, and the backoff state instead, which is equivalent and avoids adding a derived value to the carrier ProB has to enumerate.

`reconnected` is the shared `threading.Event` (`ClientRegistry._reconnected`): set by whichever actor's dial is the one that turns `linked` from `clear` to `set`, consumed by the replicator's wait. `wokenEarly` records what `wait_for_reconnect`'s own boolean return value records — whether the wait was broken by the event (`set`) or ran out the clock (`clear`) — the exact fact I6 is stated over. `reachable` and `sendOk` are the environment's own truth: whether a dial would succeed right now, and, independently, whether an already-connected send would succeed right now (a wedged or dead-peer failure is a *send-time* event, not a dial-time one, so the two are modelled as separate flags rather than folded into one three-way display-process type). `regLock` is `ClientRegistry._lock`. `discBackoff` and `wedgeBackoff` are the two independently-tuned counters of §6.2, abstracted from seconds to a step count; `lInterval` is `DisplayLiveness`'s current pacing.

Modelling `reachable` and `sendOk` as two independent flags rather than one three-valued display-process type (as `hub_replicator.tex`'s `DPROC` does) is deliberate: that model needed `dstuck` versus `ddead` to route to two different recoveries it fully derives; this model only needs to know *whether* a connected-but-misbehaving send happened at all, because the recovery itself (`rhealing`) is the already-proven machinery, entered and left in one step.

5 Initialization

<i>Init</i>	
<i>Link'</i>	
<i>live'</i>	$\emptyset$
<i>dirty'</i>	$\emptyset$
<i>shown'</i>	$\emptyset$
<i>linked'</i>	<i>clear</i>
<i>reconnected'</i>	<i>clear</i>
<i>wokenEarly'</i>	<i>clear</i>
<i>reachable'</i>	<i>clear</i>
<i>sendOk'</i>	<i>set</i>
<i>regLock'</i>	<i>free</i>
<i>rphase'</i>	<i>ridle</i>
<i>discBackoff'</i>	0
<i>wedgeBackoff'</i>	0
<i>lInterval'</i>	<i>pdisc</i>

A single initial state: nothing live, nothing held, no display connected — `DISCONNECTED`, the quiet healthy idle Hub of §4's own table — both counters at their base, the prober already at its relaxed pace, no lock held.

6 Content Operations

An agent's push and withdrawal are content-owner actions on `HubDisplay`. Neither mentions `linked`, `reachable`, `regLock`, or `rphase`: no state of the display or of either dialing actor can disable them, exactly the “the Hub never controls the display” table of §4 restated as an operation guard — or rather, the absence of one.

PushContent

$\Delta Link$

$s? : SCENE$

$live' = live \cup \{s?\}$
 $dirty' = dirty \cup \{s?\}$
 $shown' = shown \setminus \{s?\}$
 $linked' = linked$
 $reconnected' = reconnected$
 $wokenEarly' = wokenEarly$
 $reachable' = reachable$
 $sendOk' = sendOk$
 $regLock' = regLock$
 $rphase' = rphase$
 $discBackoff' = discBackoff$
 $wedgeBackoff' = wedgeBackoff$
 $lInterval' = lInterval$

A scene that becomes live is marked owed and, if it had been believed shown, that belief is withdrawn — the new content has not been delivered yet, whatever the connection state.

WithdrawContent

$\Delta Link$

$s? : SCENE$

$s? \in live$
 $live' = live \setminus \{s?\}$
 $dirty' = dirty \setminus \{s?\}$
 $shown' = shown \setminus \{s?\}$
 $linked' = linked$
 $reconnected' = reconnected$
 $wokenEarly' = wokenEarly$
 $reachable' = reachable$
 $sendOk' = sendOk$
 $regLock' = regLock$
 $rphase' = rphase$
 $discBackoff' = discBackoff$
 $wedgeBackoff' = wedgeBackoff$
 $lInterval' = lInterval$

Disposing content before any display connects is the design's **HELD** $\rightarrow$ **DISCONNECTED** transition (§4's table); nothing in this operation depends on **linked**, so it applies identically whether connected or not.

7 Environment

The display's own reachability and send-health change independently of the Hub, exactly as `hub_replicator.tex`'s `DisplayWedge` and `DisplayCrash` model a display failing on its own. These are the *only* operations that ever write a new value to **reachable** or **sendOk** — the textual fact I5 rests on.

GoReachable

$\Delta Link$

reachable = *clear*
reachable' = *set*
live' = *live*
dirty' = *dirty*
shown' = *shown*
linked' = *linked*
reconnected' = *reconnected*
wokenEarly' = *wokenEarly*
sendOk' = *sendOk*
regLock' = *regLock*
rphase' = *rphase*
discBackoff' = *discBackoff*
wedgeBackoff' = *wedgeBackoff*
lInterval' = *lInterval*

GoUnreachable

$\Delta Link$

reachable = *set*
reachable' = *clear*
live' = *live*
dirty' = *dirty*
shown' = *shown*
linked' = *linked*
reconnected' = *reconnected*
wokenEarly' = *wokenEarly*
sendOk' = *sendOk*
regLock' = *regLock*
rphase' = *rphase*
discBackoff' = *discBackoff*
wedgeBackoff' = *wedgeBackoff*
lInterval' = *lInterval*

GoSendOk

$\Delta Link$

sendOk = *clear*
sendOk' = *set*
live' = *live*
dirty' = *dirty*
shown' = *shown*
linked' = *linked*
reconnected' = *reconnected*
wokenEarly' = *wokenEarly*
reachable' = *reachable*
regLock' = *regLock*
rphase' = *rphase*
discBackoff' = *discBackoff*
wedgeBackoff' = *wedgeBackoff*
lInterval' = *lInterval*

GoSendFail

$\Delta Link$

sendOk = set
sendOk' = clear
live' = *live*
dirty' = *dirty*
shown' = *shown*
linked' = *linked*
reconnected' = *reconnected*
wokenEarly' = *wokenEarly*
reachable' = *reachable*
regLock' = *regLock*
rphase' = *rphase*
discBackoff' = *discBackoff*
wedgeBackoff' = *wedgeBackoff*
lInterval' = *lInterval*

8 The Replicator

The replicator dials only when it has something to deliver, exactly as the real worker only calls `.get()` from inside a drained cycle. A dial that finds the display reachable either completes an ordinary already-connected cycle or — if the registry was not connected a moment ago — is the fresh connect that reconciles every held scene in the very same step, discharging I2 by construction rather than by a separate pass.

RDialOk

$\Delta Link$

rphase = *ridle*
dirty $\neq \emptyset$
reachable = set
linked' = set
(*linked* = clear $\Rightarrow$ (*reconnected'* = set $\wedge$ *dirty'* = *dirty* $\cup$ *live*))
(*linked* = set $\Rightarrow$ (*reconnected'* = *reconnected* $\wedge$ *dirty'* = *dirty*))
discBackoff' = 0
rphase' = *rsending*
live' = *live*
shown' = *shown*
wokenEarly' = *wokenEarly*
reachable' = *reachable*
sendOk' = *sendOk*
regLock' = *regLock*
wedgeBackoff' = *wedgeBackoff*
lInterval' = *lInterval*

A dial that finds nothing listening is exactly today's `RuntimeError` from `ClientRegistry.get()`: no content is touched (nothing is lost, `dirty` is untouched), `linked` drops, the disconnected backoff advances toward its own cap, and the replicator moves to the wait. This is the one and only operation that advances `discBackoff`.

RDialFail

$\Delta Link$

```
rphase = ridle
dirty ≠ ∅
reachable = clear
linked' = clear
(discBackoff < 3 ⇒ discBackoff' = discBackoff + 1)
(discBackoff = 3 ⇒ discBackoff' = discBackoff)
rphase' = rwaitdisc
regLock' = free
live' = live
dirty' = dirty
shown' = shown
reconnected' = reconnected
wokenEarly' = wokenEarly
reachable' = reachable
sendOk' = sendOk
wedgeBackoff' = wedgeBackoff
lInterval' = lInterval
```

The cap is 3 here, standing in for the design's 120s ceiling; `regLock' = free` is not a passthrough but the load-bearing fact of §6.3 stated at the one place it is decided — entering the wait never takes the lock. `RDialFail`'s guard is disjoint from `RDialOk`'s (`reachable` has exactly two values), so exactly one of the two is ever the successor from `ridle` with work pending.

The wait is broken one of two ways. `RWaitWoken` models `ClientRegistry.wait_for_reconnect` returning `True`: the event had been set by *either* actor's successful dial, so the replicator retries at once, without paying any more of the current delay.

RWaitWoken

$\Delta Link$

```
rphase = rwaitdisc
reconnected = set
rphase' = ridle
reconnected' = clear
wokenEarly' = set
live' = live
dirty' = dirty
shown' = shown
linked' = linked
reachable' = reachable
sendOk' = sendOk
regLock' = regLock
discBackoff' = discBackoff
wedgeBackoff' = wedgeBackoff
lInterval' = lInterval
```

`RWaitTimeout` models the same call returning `False`: the full current delay elapsed with no reconnect. `wait_for_reconnect` clears the event on *either* exit, matching `Event.wait` followed unconditionally by `.clear()` in the real code, so both operations agree on that one point and differ only in `wokenEarly`.

RWaitTimeout

$\Delta Link$

$rphase = rwaitdisc$
 $rphase' = ridle$
 $reconnected' = clear$
 $wokenEarly' = clear$
 $live' = live$
 $dirty' = dirty$
 $shown' = shown$
 $linked' = linked$
 $reachable' = reachable$
 $sendOk' = sendOk$
 $regLock' = regLock$
 $discBackoff' = discBackoff$
 $wedgeBackoff' = wedgeBackoff$
 $lInterval' = lInterval$

Once connected, a delivery either succeeds — everything held is shown, and the wedged-display backoff resets, since a clean cycle is unambiguous in a way “hasn’t misbehaved again yet” is not (§6.3) — or the send itself fails against an already-open connection, which is the *different* condition this design is careful never to confuse with a failed dial.

RSendCycle

$\Delta Link$

$rphase = rsending$
 $sendOk = set$
 $dirty' = \emptyset$
 $shown' = shown \cup dirty$
 $wedgeBackoff' = 0$
 $rphase' = ridle$
 $live' = live$
 $linked' = linked$
 $reconnected' = reconnected$
 $wokenEarly' = wokenEarly$
 $reachable' = reachable$
 $sendOk' = sendOk$
 $regLock' = regLock$
 $discBackoff' = discBackoff$
 $lInterval' = lInterval$

RSendFail

$\Delta Link$

```
rphase = rsending
sendOk = clear
(wedgeBackoff < 1  $\Rightarrow$  wedgeBackoff' = wedgeBackoff + 1)
(wedgeBackoff = 1  $\Rightarrow$  wedgeBackoff' = wedgeBackoff)
rphase' = rhealing
live' = live
dirty' = dirty
shown' = shown
linked' = linked
reconnected' = reconnected
wokenEarly' = wokenEarly
reachable' = reachable
sendOk' = sendOk
regLock' = regLock
discBackoff' = discBackoff
lInterval' = lInterval
```

The cap is 1 here, standing in for the design’s 2.0s ceiling — deliberately much lower than `discBackoff`’s cap of 3, the concrete gap I4 is stated over. `RSendFail` is the only operation that advances `wedgeBackoff`, and it never touches `discBackoff`; `RDialFail` is the only operation that advances `discBackoff`, and it never touches `wedgeBackoff`. That disjointness is I7, established by inspection exactly as `hub_replicator.tex`’s I1 is (“no mutator operation mentions `clientLock`”): there is no third operation and no shared branch that could touch both.

`RHeal` abstracts the reap/respawn/reconnect/re-mark recovery `hub_replicator.tex` already proves in full (its `Reap`, `Ensure`, `Remark`, `Reconn`): here it is one atomic step, entered only from `rhealing` and always eventually leaving it, touching nothing this spec cares about beyond returning control to the cycle.

RHeal

$\Delta Link$

```
rphase = rhealing
rphase' = ridle
live' = live
dirty' = dirty
shown' = shown
linked' = linked
reconnected' = reconnected
wokenEarly' = wokenEarly
reachable' = reachable
sendOk' = sendOk
regLock' = regLock
discBackoff' = discBackoff
wedgeBackoff' = wedgeBackoff
lInterval' = lInterval
```

9 The Liveness Prober

`DisplayLiveness` probes continuously and independently of the replicator’s own cycle — it has no notion of “dirty content,” only of whether the display answers. Its dial goes through the same registry lock the replicator’s does, so its guard is exactly `regLock = free`; under the corrected design that guard is never actually contended, because nothing ever holds the lock across a wait (§Fidelity makes this concrete). A successful probe can be the very dial that turns `linked` from `clear` to `set` and reconciles held content — “ANY successful `ClientRegistry.get()`” in the design’s own words, not only the replicator’s.

LDialOk

$\Delta Link$

regLock = free
reachable = set
linked' = set
(*linked* = clear $\Rightarrow$ (*reconnected'* = set $\wedge$ *dirty'* = *dirty* $\cup$ *live*))
(*linked* = set $\Rightarrow$ (*reconnected'* = *reconnected* $\wedge$ *dirty'* = *dirty*))
Interval' = *pconn*
live' = *live*
shown' = *shown*
wokenEarly' = *wokenEarly*
reachable' = *reachable*
sendOk' = *sendOk*
regLock' = *regLock*
rphase' = *rphase*
discBackoff' = *discBackoff*
wedgeBackoff' = *wedgeBackoff*

LDialFail

$\Delta Link$

regLock = free
reachable = clear
linked' = clear
Interval' = *pdisc*
live' = *live*
dirty' = *dirty*
shown' = *shown*
reconnected' = *reconnected*
wokenEarly' = *wokenEarly*
reachable' = *reachable*
sendOk' = *sendOk*
regLock' = *regLock*
rphase' = *rphase*
discBackoff' = *discBackoff*
wedgeBackoff' = *wedgeBackoff*

Neither operation is guarded by **rphase**: the prober runs regardless of whatever cycle the replicator is in, which is the whole point of it being a genuinely independent second actor rather than a second phase of the same worker.

10 Invariants

None of the eight properties below is placed in **Link**'s own predicate, for the same reason **hub_replicator.tex** keeps its six out of **Repl**: a predicate there would silently disable any operation that would break it, masking exactly the defect the model exists to catch.

I1 — content never lost while disconnected. Every live scene is either owed or already believed delivered:

$$\forall s : SCENE \mid s \in live \bullet s \in dirty \vee s \in shown.$$

No operation removes a scene from **dirty** without also adding it to **shown** (**RSendCycle**) or removing it from **live** in the same step (**WithdrawContent**); no operation ever adds to **live** without adding the same scene to **dirty** (**PushContent**). The claim holds unconditionally, not only while **linked** = **clear**: a scene mid-cycle, not yet delivered, is still covered because it is still in **dirty**.

I2 — reconcile exactly once. `dirty' = dirty  $\cup$  livefiresonlyinsideRDialOkandLDialOk, andonlyundertheguardlink`
*---i.e. onlyonthe stepthatturnstheregistryfromdisconnectedtoconnected. Because linkedisonesharedboolean, atmostoneofthe
it ispartofthesameatomicstepthatperformsthetransition. Aheldscenebelievedalreadyshownisthesharpestwitness :
onlythereconcilecanputasceneintodirtywithoutfirsttakingitoutof shown, sinceeveryotherproducerof dirty(PushContent)rem*

I3 — neither actor blocks or spins on a missing display. Every dial (`RDialOk`, `RDialFail`, `LDialOk`, `LDialFail`) is one Z operation and therefore resolves in one bounded step by construction
--- there is no schema in this model whose enabling condition can remain true indefinitely without an intervening state change, unlike a raw `time.sleep` which is not a state transition at all. `lInterval` is typed `SPEED`, so the prober's pacing is always exactly one of the two named values; the replicator's own interval between retries is governed by `discBackoff` alone (`RWaitTimeout`'s only enabling condition is `rphase = rwaitdisc`, entered only via `RDialFail`, which is the only place `discBackoff` advances) --- there is no third, unnamed pacing value either actor can fall into.

I4 — the disconnected backoff reaches its own, higher, slow steady state.

(discBackoff = 3)

is reachable under sustained disconnection alone (no send ever attempted). This is the specific defect this design fixes, restated as a reachability property rather than an “eventually bounded” one: §Fidelity shows that collapsing the two counters makes this exact state unreachable, because nothing then advances `discBackoff` past whatever the shared, lower-capped counter tops out at.

I5 — the Hub never issues a process-control call to the display. No operation other than `GoReachable` ever writes `set` to `reachable` where `reachable = clear` beforehand --- inspection of every schema above confirms each either leaves `reachable` unchanged or is `GoReachable/GoUnreachabl` themselves, the two environment operations standing in for a human running lux display start/stop or the OS service supervisor. This is the absolute, grep-checkable form of the operator's ruling (§2 item 1 of the design), restated as the claim that the model's own carrier of “a display becomes reachable” has exactly one non-environment writer: none.

I6 — the early-wake mechanism is present, reachable, and load-bearing.

(wokenEarly = set)

is reachable: the replicator's wait *can* be broken by the shared event, not only by the full delay elapsing. §Fidelity shows this is exactly the property that a raw blocking sleep --- `RWaitWoken` deleted --- makes unreachable, reproducing “up-to-full-delay blindness” precisely: the mechanism this proves present is also the mechanism whose absence is observably worse.

This is not the whole of design §12's I6, and the gap is worth stating plainly rather than papering over. The design's own words are that a reconnect breaks the replicator out “within one `DisplayLiveness` probe tick,” a real-time bound. `RWaitTimeout`'s only guard in this model is `rphase = rwaitdisc`: at a state where `reconnected = set`, both `RWaitWoken` and `RWaitTimeout` are enabled, and the model admits a legal trace that takes the timeout exit *despite* the wake signal being available. An untimed Z model has no clock against which “within one probe tick” could even be stated, let alone proved; reachability of `wokenEarly = set` shows the early-exit path exists and matters, but it does not show the replicator *always* takes it the instant the signal is available. The actual real-time promptness claim rests on two facts this model proves true by construction and one it does not model at all: I8 below establishes that the wait never holds the registry lock, so the prober's own dial is never blocked from setting the signal; §6.4's 5-second `_DISCONNECTED_PROBE_INTERVAL` bounds how long the prober can go between attempts (a real-time fact about the implementation, not a Z proposition); and the actual scheduling fairness between `RWaitWoken` and `RWaitTimeout` at the moment both are enabled is a property of the real thread scheduler, not of this specification. A liveness variant of this property --- “if parked with the signal set, the wait is broken by the wake and not by repeated, signal-ignoring timeouts” --- is expressible as an LTL

check under weak fairness on RWaitWoken; it was considered and set aside for this revision (§Fidelity) as not adding proof strength proportionate to its cost, since the honest restatement above already draws the line the design document itself needed redrawn.

I7 — the two backoffs never advance together. RDialFail is the only operation whose effect changes discBackoff, and its frame leaves wedgeBackoff unchanged; RSendFail is the only operation whose effect changes wedgeBackoff, and its frame leaves discBackoff unchanged. No operation instance can therefore advance both counters in one step --- established by inspection, exactly as hub_replicator.tex establishes its own I1. The mechanism is corroborated live (not simply dead code) by the reachability of

$$(discBackoff > 0 \wedge wedgeBackoff > 0)$$

--- both counters independently nonzero, from two disjoint causes.

I8 — deadlock-freedom; the wait never holds the registry lock.

$$\neg (rphase = rwaitdisc \wedge regLock = held).$$

No operation in this specification ever sets regLock' = held: it is free at Init and every subsequent schema either leaves it unchanged or (RDialFail) explicitly re-asserts free at the one place §6.3's design decision is made. The prober's own dial (LDialOk/LDialFail) is therefore never disabled by the replicator sitting in rwaitdisc, and the full state space is free of deadlock (Verification, below). §Fidelity constructs the variant where this fails, and shows the goal above becomes reachable and the prober's dial becomes starved for as long as the replicator waits.

The scope of this claim is worth stating precisely, not left implicit. regLock = held is unreachable in *every* state of the corrected model, not merely in rwaitdisc --- every dial (RDialOk, RDialFail, LDialOk, LDialFail) is one atomic Z step, so nothing this model can observe ever holds the lock across more than the single step in which it is momentarily acquired and released. “The wait never holds the lock” is therefore true here in the degenerate sense that the lock is never held across *any* multi-step span at all; the specific hazard I8 is guarding against --- a wait that wrongly extends the hold across the whole parked interval --- is exercised only by fidelity control (c), which is the one place regLock does anything interesting in this specification. This is an honest, not an evasive, degeneracy: it reflects that ClientRegistry.get()'s real critical section genuinely is short and bounded, and the design's actual risk was always specifically about *extending* that hold across wait_for_reconnect which is exactly what fidelity control (c) targets.

This paragraph's deadlock-freedom claim is also scoped to regLock alone. hub_replicator.tex proves its own I3 --- no thread ever holds its store lock (StoreLock, guarding HubDisplay's content) and the client send lock at once --- for the *single-actor* replicator model; this specification does not re-derive a two-lock ordering across regLock and StoreLock for the two-actor case, and rests on a premise it states rather than proves: wait_for_reconnect acquires no second lock while parked. ClientRegistry._lock and StoreLock are two independent locks in the real implementation, and nothing in wait_for_reconnect's own body (§6.3) touches StoreLock at all --- but that is a fact about the implementation this model takes as a premise, not a conclusion this model's operations derive, because StoreLock does not appear in Link's carrier. An implementation review should confirm that premise holds of the shipped code, not merely of this specification.

11 Verification

Type-checking is a fuzz pass:

```
fuzz docs/display_linkage.tex
```

I1, I5, and I8's lock claim are state properties, checked as reachability of the negation (*goal NOT found* means the invariant holds everywhere reachable):

```
# Invariant 1 (must report: goal NOT found)
probccli docs/display_linkage.tex -model_check -nodead \
  -goal "card({s | s : SCENE & s : live &
    not(s : dirty) & not(s : shown)}) > 0" \
  -p DEFAULT_SETSIZE 2

# Invariant 8, the lock claim (must report: goal NOT found)
probccli docs/display_linkage.tex -model_check -nodead \
  -goal "rphase = rwaitdisc & regLock = held" \
  -p DEFAULT_SETSIZE 2
```

I2, I3, I5, and I7 are established by inspection of the operation schemas (Section 10) and corroborated live by reachability goals that must be *found* --- the mechanism is exercised, not dead:

```
# Invariant 2 corroboration (must report: goal FOUND)
probccli docs/display_linkage.tex -model_check -nodead \
  -goal "card(dirty /\ shown) > 0" -p DEFAULT_SETSIZE 2

# Invariant 3 corroboration -- both named prober speeds are live
probccli docs/display_linkage.tex -model_check -nodead \
  -goal "lInterval = pdisc" -p DEFAULT_SETSIZE 2
probccli docs/display_linkage.tex -model_check -nodead \
  -goal "lInterval = pconn" -p DEFAULT_SETSIZE 2

# Invariant 7 corroboration (must report: goal FOUND)
probccli docs/display_linkage.tex -model_check -nodead \
  -goal "discBackoff > 0 & wedgeBackoff > 0" -p DEFAULT_SETSIZE 2
```

I4 and I6 are the two properties this design exists to fix, each checked as reachability that must be *found* in the corrected model:

```
# Invariant 4 (must report: goal FOUND)
probccli docs/display_linkage.tex -model_check -nodead \
  -goal "discBackoff = 3" -p DEFAULT_SETSIZE 2

# Invariant 6 (must report: goal FOUND)
probccli docs/display_linkage.tex -model_check -nodead \
  -goal "wokenEarly = set" -p DEFAULT_SETSIZE 2
```

Deadlock freedom is the full-state-space check. Every phase has an always-available successor --- PushContent/WithdrawContent require nothing about the display, RWaitTimeout requires nothing beyond rphase = rwaitdisc, and each of sendOk's two values enables exactly one of RSendCycle/RSendFail --- so any deadlock reported would be a genuine cyclic wait, not this model terminating:

```
# Deadlock freedom (must report: no deadlock / no counterexample)
probccli docs/display_linkage.tex -model_check -p DEFAULT_SETSIZE 2
```

12 Fidelity: reproducing the design's own defects

A model that cannot reproduce the defects it is meant to guard against proves nothing. This specification guards three; each is reproduced by a small, named change to one operation, verified against a scratch copy of this file with exactly that change applied and nothing else.

(a) **Collapsing the tri-state outcome onto the shared backoff.** The design's own root-cause bug (§1, §6.1): before the fix, *every* not-connected cycle advanced the same low-capped backoff a wedged display uses, because the code could not tell ‘‘never connected’’ apart from ‘‘connected but just healed.’’ Reproduced here by changing RDialFail's effect from

advancing discBackoff (cap 3) to advancing wedgeBackoff (cap 1) instead --- the literal ‘collapse onto the shared backoff’:

```
(wedgeBackoff < 1 ⇒ wedgeBackoff' = wedgeBackoff + 1)
(wedgeBackoff = 1 ⇒ wedgeBackoff' = wedgeBackoff)
discBackoff' = discBackoff
```

in place of the two discBackoff lines shown above. The property that separates the two designs is I4 itself:

```
# must be FOUND for the correct spec, NOT FOUND for the collapsed variant
probccli <spec>.tex -model_check -nodead \
  -goal "discBackoff = 3" -p DEFAULT_SETSIZE 2
```

Run against a scratch copy with the one-line substitution above: the goal returns *not found* --- discBackoff is never touched by any operation in that variant, so it never leaves 0, while wedgeBackoff climbs under sustained disconnection alone and plateaus at its own cap of 1 --- the model’s rendering of ‘the backoff plateaus at ~2s and stays there forever,’ exactly the symptom of §1’s confirmed root cause. Reachability of wedgeBackoff = 1 under a disconnection-only trace (no RSendFail ever fired) is the positive confirmation of the same collapse:

```
# FOUND in the collapsed variant via RDialFail alone
probccli <spec>.tex -model_check -nodead \
  -goal "wedgeBackoff = 1" -p DEFAULT_SETSIZE 2
```

(b) A blocking sleep in place of the interruptible wait. The design’s own second, load-bearing correction (§6.3): a raw time.sleep(delay) would commit the replicator to sleeping out the *entire* current delay --- up to 120s once backed off --- before attempting another connect, regardless of how promptly the display actually returns. Reproduced here by deleting RWaitWoken from the operation set entirely, leaving RWaitTimeout as the only way out of rwaitdisc:

```
# RWaitWoken removed; RWaitTimeout unchanged
```

The property that separates the two designs is I6 itself:

```
# must be FOUND for the correct spec, NOT FOUND for the blocking-sleep variant
probccli <spec>.tex -model_check -nodead \
  -goal "wokenEarly = set" -p DEFAULT_SETSIZE 2
```

With RWaitWoken removed, nothing in the model ever sets wokenEarly to set --- the goal returns *not found*. Concretely: reachable can flip to set (GoReachable) and the prober can dial successfully (LDialOk, itself unaffected by the deletion) and set reconnected, all while rphase = rwaitdisc --- but with no RWaitWoken to consume that signal, the replicator remains parked until RWaitTimeout eventually fires regardless, discovering the display no sooner than a display that never reconnected at all would have been retried. This is the model’s rendering of ‘a display connect not promptly breaking the replicator out’ --- exactly the second required symptom, and exactly the failure mode §6.3’s own worked example describes (‘a user who runs lux display start right after the Hub has climbed deep into its backoff would see held content stay blank for up to two minutes’).

(c) Holding the registry lock across the wait (bonus control for I8). Not one of the two symptoms the mission requires, but the most direct way to exhibit what §6.3’s own prose warns the lock-freedom property prevents. Reproduced by two changes: RDialFail sets regLock’ = held instead of free, and both RWaitWoken and RWaitTimeout gain regLock’ = free in place of their passthrough. The distinguishing goal is I8’s own:

```
# must be NOT FOUND for the correct spec, FOUND for the lock-holding variant
probccli <spec>.tex -model_check -nodead \
  -goal "rphase = rwaitdisc & regLock = held" -p DEFAULT_SETSIZE 2
```

In the lock-holding variant this goal is reached immediately after the first `RDialFail`, and for as long as the replicator remains parked, `LDialOk/LDialFail`'s shared guard `regLock = free` is false --- the prober is starved from dialling at all, which in turn starves reconnected of the one signal that could wake the replicator early, so the only escape becomes `RWaitTimeout`: precisely the mechanism by which '‘holding the lock across the wait’' would defeat the very promptness I6 exists to guarantee, stated here as a positive account of *why* §6.3 insists on `wait_for_reconnect` not taking `ClientRegistry._lock`, not merely that it must not. This variant does not produce a literal ProB deadlock --- `RWaitTimeout` remains an always-enabled escape, and the content operations never consult `regLock` at all, so the state space stays live -- it produces exactly the starvation the reachability goal states, which is the honest and sufficient demonstration of the claim; a spec that manufactured a true deadlock here would need to also remove `RWaitTimeout`, which is a different, unrelated defect from the one this control targets.